Comparative Analysis of Algorithm Efficiency vs. Energy Consumption

Activity 1

Problem Explanation

This task asks us to take a large dataset and process it in two different styles, then compare how long each style takes to run. The first style is the traditional way: use a Vector to hold the data and use nested for loops to go through it. The second style is the modern way: use Java's Streams API, and then Parallel Streams, which can split work across multiple CPU cores. After timing both approaches, we are expected to explain why a slower algorithm, meaning one with higher time complexity, is not just slower on paper, but also means the server's processor stays busy longer, generates more heat, and pulls more electricity, since energy use is roughly tied to how long the CPU is working at full load.

To make the nested loop genuinely meaningful, and not just a single flat loop, the task is built around checking how many numbers in a large dataset are prime numbers. Checking if a single number is prime needs a loop of its own, so when every number in a big dataset is checked, the result is naturally an outer loop plus an inner loop, which is the nested structure the assignment is asking for.

Java Code

```
import java.util.Vector;
import java.util.List;
import java.util.ArrayList;
import java.util.Random;

public class AlgorithmEfficiencyTest {

    // Checks if a number is prime using simple trial division
    public static boolean isPrime(int number) {
        if (number &lt; 2) {
            return false;
        }
        for (int i = 2; i * i &lt;= number; i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int datasetSize = 500000;
        Random random = new Random();

        // Creating the dataset using a Vector
        Vector&lt;Integer&gt; dataset = new Vector&lt;&gt;();
        for (int i = 0; i &lt; datasetSize; i++) {
            dataset.add(random.nextInt(1000000) + 1);
        }
        // A normal List copy is needed for the Streams part
```

```
        List&lt;Integer&gt; streamDataset = new ArrayList&lt;&gt;(dataset);

        System.out.println("Dataset size: " + datasetSize);
        System.out.println();

        // Approach 1: Traditional nested for loop with Vector
        long startTime1 = System.nanoTime();
        int primeCount1 = 0;
        for (int i = 0; i &lt; dataset.size(); i++) {
            int number = dataset.get(i);
            if (isPrime(number)) {
                primeCount1++;
            }
        }
        long endTime1 = System.nanoTime();
        long duration1 = (endTime1 - startTime1) / 1000000;

        System.out.println("Traditional nested loop + Vector");
        System.out.println("Primes found: " + primeCount1);
        System.out.println("Time taken: " + duration1 + " ms");
        System.out.println();

        // Approach 2: Streams API (sequential)
        long startTime2 = System.nanoTime();
        long primeCount2 = streamDataset.stream()
                .filter(AlgorithmEfficiencyTest::isPrime)
                .count();
        long endTime2 = System.nanoTime();
        long duration2 = (endTime2 - startTime2) / 1000000;

        System.out.println("Streams API (sequential)");
        System.out.println("Primes found: " + primeCount2);
        System.out.println("Time taken: " + duration2 + " ms");
        System.out.println();

        // Approach 3: Parallel Streams
        long startTime3 = System.nanoTime();
        long primeCount3 = streamDataset.parallelStream()
                .filter(AlgorithmEfficiencyTest::isPrime)
                .count();
        long endTime3 = System.nanoTime();
        long duration3 = (endTime3 - startTime3) / 1000000;

        System.out.println("Parallel Streams");
        System.out.println("Primes found: " + primeCount3);
        System.out.println("Time taken: " + duration3 + " ms");
    }
}
```

Expected Output

The exact numbers will change every time the program runs, since the dataset is randomly generated and timing depends on the machine it runs on, but the pattern will look something like this:

```
Dataset size: 500000

Traditional nested loop + Vector
Primes found: 41538
```

```
Time taken: 612 ms

Streams API (sequential)
Primes found: 41538
Time taken: 598 ms

Parallel Streams
Primes found: 41538
Time taken: 187 ms
```

The prime count stays the same across all three approaches, since they are checking the same data with the same logic. Only the timing changes.

Code Explanation

The isPrime method checks whether a number can be divided evenly by anything other than 1 and itself. It only checks divisors up to the square root of the number, because if a number has a factor larger than its square root, it must also have a matching factor smaller than the square root, so checking beyond that point is wasted work.

The dataset itself is built using a Vector, as the assignment specifically requires. A Vector works like an ArrayList, but every method on it is synchronized, which means only one thread can access it at a time. This makes it thread-safe but slower under concurrent access, which becomes relevant later when comparing it against parallel streams.

The first approach uses a plain for loop to walk through the Vector one item at a time, and for each item it calls isPrime, which runs its own loop internally. That combination, an outer loop over the dataset and an inner loop checking divisibility, is the nested loop structure the assignment asks for.

The second approach uses the Streams API. Streams let the code describe what should happen, filter for primes and count them, rather than writing the loop by hand. Internally, Java still walks through the data, but it does so sequentially, one item after another, which makes its speed comparable to the manual loop.

The third approach uses parallelStream, which splits the dataset into chunks and processes those chunks on different CPU cores at the same time, then combines the results at the end. This is why it usually finishes noticeably faster on a multi-core machine, since the work is genuinely happening simultaneously rather than one item after another.

Time Complexity, Heat, and Electricity Cost

The isPrime check runs in roughly O(sqrt(n)) time for a number of size n. Since this check runs for every item in the dataset, both the traditional loop and the sequential stream run in about O(size × sqrt(n)) time overall. Parallel streams perform about the same total amount of work, but that work is divided across multiple cores, so the wall-clock time drops even though the total CPU work stays roughly the same.

This connects directly to how a real server behaves. A CPU core running at high load draws more electrical current than a core sitting idle, and that extra current is released as heat that the server has to get rid of. Two factors mainly drive the electricity bill and the cooling load: how much total work the CPU does, and how long it stays under that load. An algorithm with poor time complexity forces the CPU to remain busy for longer to finish the same job, which means more heat produced, more strain on the data center's cooling system, and a higher electricity bill, both from the computation itself and from the air conditioning needed to keep the servers from overheating.

This is why data centers treat algorithmic efficiency as a cost issue and not only a speed issue. Parallel processing does not reduce the total amount of work being done, but if it finishes the job faster, the server can return to idle sooner, which can help overall energy use, provided the extra cores being used were not otherwise sitting idle and unused for something else.